

GOLD EA ARCHITECTURE & SPECIFICATION

FULL 9-STRATEGY MULTI-TIMEFRAME SYSTEM SPECIFICATION (XAUUSD) — VERSION 2.0

AI BUILD PROMPT INSTRUCTIONS: This document contains the end-to-end technical specification required to program an Expert Advisor (EA) in MQL5/C++. It covers the core multi-timeframe breakout engine, risk management controls, dynamic percentage sizing, prop firm safeguards, and the exact data structures required to execute **all 9 sub-strategies concurrently**.

1. System Overview & Core Principles

The system is a fully automated Expert Advisor designed specifically for Gold (XAUUSD). It operates on a **Multi-Timeframe Breakout Engine** using market structure and fractal swing highs and lows. The architecture incorporates **9 distinct sub-strategies running concurrently** within a single EA instance. Each sub-strategy shares the same core breakout execution logic but operates with unique parameter sets (timeframes, offsets, fractal sensitivities, and risk weightings).

Core Design Rules:

- **Strict Risk Protections:** Zero Martingale, zero grid, zero averaging down. Every trade enters with a defined Stop Loss.
- **Dynamic Percentage Scaling:** All distances (entry offsets, stop loss, take profit, trailing stop, salvage trigger) are calculated as a percentage of current market price (e.g., Ask/Bid), making the strategy fully adaptive to Gold price fluctuations without manual pip recalculations.
- **Prop Firm Compliance:** Features strict daily and overall equity drawdown hard caps, trade randomizer offsets, and virtual order expirations.

2. Full Input Parameter Specifications

Parameter Name	Type	Default Value	Description & Logic
MagicNumber	ulong	108800	Base magic number. Sub-strategy i uses MagicNumber + i.
TradeComment	string	Gold X9 CapFree	Comment string attached to executed trades.
LotSizingMethod	Enum	Tiered (2)	1 = Fixed Lot, 2 = Tiered Scale (1-100), 3 = Risk Multiplier.
TieredLotsValue	int	15	Tiered risk allocation level between 1 and 100.
FixedLotSize	double	0.10	Manual lot size if LotSizingMethod = Fixed.
RiskScaleFactor	double	1.0	Risk scaling multiplier applied across all lot calculations.
EnableRandomizer	bool	true	Applies micro-offsets (+/- 0.1-0.3 pts) to comply with prop firm limits.
DailyDrawdownCapPct	double	2.5%	Hard stop: closes all positions if daily equity drops > 2.5%.
MaxDrawdownCapPct	double	8.0%	Hard stop: halts EA if peak-to-trough equity drawdown > 8.0%.

3. Base Strategy Parameters (Strategy #9 Standard)

The table below details the exact default parameter settings for **Sub-Strategy #9**, which serves as the base baseline model extracted directly from the video tutorials.

Setting Name	Value	Explanation & Formulas
FractalTimeframe	M15 / H1	Timeframe scanned for swing high and swing low fractal formation.
FractalLeft / Right	5 / 5 bars	Requires 5 lower highs to the left and 5 lower highs to the right for a valid high.
MinFractalClearancePct	0.02%	Min distance from fractal high/low required before placing pending order.
BuyEntryOffsetPct	-0.08%	Placed BELOW swing high: BuyStop Price = SwingHigh * (1.0 - 0.0008).
SellEntryOffsetPct	0.08%	Placed ABOVE swing low: SellStop Price = SwingLow * (1.0 + 0.0008).
StopLossPct	2.0%	Stop Loss distance calculated as percentage of current price.
TakeProfitPct	0.7%	Take Profit distance calculated as percentage of current price.
BreakEvenTriggerPct	0.10%	Profit threshold % required to activate Break-Even lock.
BreakEvenLockPct	0.025%	Profit locked in above entry price upon BE trigger.
TrailingTriggerPct	0.20%	Profit threshold % required to activate Trailing Stop.
TrailingDistancePct	0.20%	Distance percentage maintained behind market price.
MaxTrailingPct	1.00%	Cap on maximum trailed distance to allow full TP targets.
SalvageTriggerPct	1.00%	Floating loss % threshold (halfway to SL) to activate Salvage Mode.
SalvageTargetPct	-0.15%	Moves TP into minor loss territory (-0.15%) to cut loss on bounce.
PendingExpiryHours	15.0 hrs	Pending orders automatically cancelled after 15 hours.
RiskWeight	0.055	Relative risk weight assigned to Sub-Strategy #9.

4. Complete Multi-Strategy Engine (9 Sub-Strategies)

To achieve maximum consistency across varying Gold market conditions, the EA manages **9 parallel sub-strategies** simultaneously. They share a single order execution pipeline but maintain independent state machines, pending orders, and active positions.

4.1 Multi-Strategy Struct & Data Architecture (MQL5)

```
// --- MQL5 MULTI-STRATEGY PARAMETER STRUCTURE ---
struct StrategyConfig {
    int      strategyId;          // Strategy Index (0 to 8)
    ulong    magicNumber;        // BaseMagic + strategyId
    ENUM_TIMEFRAMES timeframe;    // Execution & Fractal Timeframe
    int      fractalLeft;        // Left bars count (e.g., 3, 5, 7)
    int      fractalRight;       // Right bars count (e.g., 3, 5, 7)
    double   buyOffsetPct;       // Buy Entry Offset %
    double   sellOffsetPct;      // Sell Entry Offset %
    double   stopLossPct;        // Stop Loss %
    double   takeProfitPct;      // Take Profit %
    double   breakEvenTriggerPct; // Break-Even Activation %
    double   breakEvenLockPct;   // Break-Even Locked Profit %
    double   trailingTriggerPct;  // Trailing Activation %
    double   trailingDistPct;     // Trailing Distance %
    double   salvageTriggerPct;   // Salvage Mode Activation % (Loss Threshold)
    double   salvageTargetPct;   // Salvage TP Target %
    double   riskWeight;         // Relative Risk Weighting (0.0 to 1.0)
    bool     enabled;            // Toggle switch for individual strategy
};

StrategyConfig SubStrategies[9]; // Array storing all 9 sub-strategies
```

4.2 Initialization & Parameter Matrix for Strategies #1 through #9

Sub-Strategy #9 serves as the baseline setup. Strategies #1 through #8 are diversified variations spanning different timeframes, fractal sensitivities, and breakout offsets. The EA code initializes the 9 strategies as follows:

```
void InitMultiStrategyEngine(ulong baseMagic) {
    // Strategy #9 (Base Benchmark Strategy)
    SubStrategies[8].strategyId = 9; SubStrategies[8].magicNumber = baseMagic + 8;
    SubStrategies[8].timeframe = PERIOD_M15; SubStrategies[8].fractalLeft = 5; SubStrategies[8].fractalRight = 5;
    SubStrategies[8].buyOffsetPct = -0.08; SubStrategies[8].sellOffsetPct = 0.08;
    SubStrategies[8].stopLossPct = 2.0; SubStrategies[8].takeProfitPct = 0.7;
    SubStrategies[8].breakEvenTriggerPct = 0.10; SubStrategies[8].breakEvenLockPct = 0.025;
    SubStrategies[8].trailingTriggerPct = 0.20; SubStrategies[8].trailingDistPct = 0.20;
    SubStrategies[8].salvageTriggerPct = 1.00; SubStrategies[8].salvageTargetPct = -0.15;
    SubStrategies[8].riskWeight = 0.055; SubStrategies[8].enabled = true;

    // Strategies #1 to #8 (Diversified Multi-Timeframe Presets)
    ENUM_TIMEFRAMES tf_list[8] = {PERIOD_M1, PERIOD_M5, PERIOD_M5, PERIOD_M15, PERIOD_M15, PERIOD_M30, PERIOD_H1, PERIOD_D1};
    int left_list[8] = {3, 3, 5, 5, 7, 5, 5, 7};
    double offsets[8] = {-0.04, -0.05, -0.06, -0.08, -0.10, -0.12, -0.15, -0.18};

    for(int i = 0; i < 8; i++) {
        SubStrategies[i].strategyId = i + 1;
        SubStrategies[i].magicNumber = baseMagic + i;
        SubStrategies[i].timeframe = tf_list[i];
        SubStrategies[i].fractalLeft = left_list[i];
        SubStrategies[i].fractalRight = left_list[i];
        SubStrategies[i].buyOffsetPct = offsets[i];
        SubStrategies[i].sellOffsetPct = -offsets[i];
        SubStrategies[i].stopLossPct = 1.5 + (i * 0.15);
        SubStrategies[i].takeProfitPct = 0.5 + (i * 0.08);
        SubStrategies[i].breakEvenTriggerPct = 0.08 + (i * 0.01);
        SubStrategies[i].breakEvenLockPct = 0.02;
        SubStrategies[i].trailingTriggerPct = 0.15 + (i * 0.02);
        SubStrategies[i].trailingDistPct = 0.15;
        SubStrategies[i].salvageTriggerPct = SubStrategies[i].stopLossPct * 0.5;
        SubStrategies[i].salvageTargetPct = -0.10;
        SubStrategies[i].riskWeight = 0.05 + (i * 0.005);
        SubStrategies[i].enabled = true;
    }
}
```

4.3 Main Execution Loop (`OnTick` Iteration)

```
void OnTick() {
    // 1. Global Equity Drawdown Safety Check
    if(CheckGlobalDrawdownCaps()) return;

    // 2. Iterate through all 9 Sub-Strategies
    for(int i = 0; i < 9; i++) {
        if(!SubStrategies[i].enabled) continue;

        // A. Scan Swing Points & Manage Pending Orders for Strategy i
        ProcessStrategyBreakouts(SubStrategies[i]);

        // B. Manage Active Positions (BE, Trailing Stop, Salvage) for Strategy i
        ManageActivePositions(SubStrategies[i]);
    }
}
```

5. Detailed Algorithmic Workflows

A. Swing High & Swing Low Detection:

- Candidate bar is index `Bar = FractalLeft + 1` (e.g. bar 6).
- Validate that all `FractalRight` bars (1 to 5) and `FractalLeft` bars (7 to 11) have Highs lower than candidate bar (for Swing High) or Lows higher (for Swing Low).
- Confirm price distance from fractal point meets `MinFractalClearancePct` (0.02%).

B. Order Placement & Replacement Logic:

- Buy Stop placed at `SwingHigh \* (1.0 + BuyOffsetPct)` (where negative offset places order slightly below swing high).
- Sell Stop placed at `SwingLow \* (1.0 + SellOffsetPct)`.
- Check `DuplicatePointGap` (5 points) against existing active orders to prevent stacking.
- If max pending order count is reached, delete the worst-priced pending order if new order is superior.

C. Active Position Management:

- **Break-Even Lock:** Triggered when floating profit $\geq$ `BreakEvenTriggerPct`. SL is moved to `EntryPrice + BreakEvenLockPct`.
- **Trailing Stop:** Triggered when profit $\geq$ `TrailingTriggerPct`. Trails SL at `TrailingDistancePct` behind price until `MaxTrailingPct` is hit.
- **Salvage Mode:** If floating loss reaches `SalvageTriggerPct` (1% loss, halfway to 2% SL), TP is shifted to `EntryPrice - SalvageTargetPct` (-0.15%) to exit on retracement and prevent full SL hit.